

Тема 13. Паттерны проектирования

5. Структурные паттерны

6. Паттерны поведения

7. Порождающие паттерны

Структурные паттерны

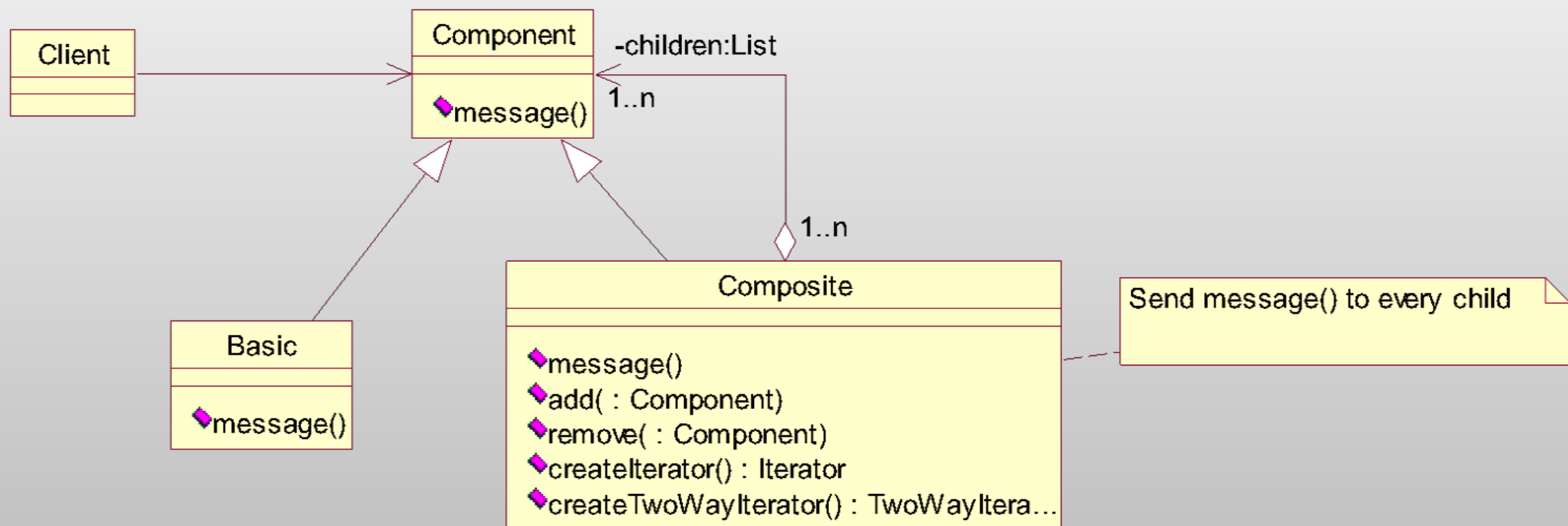
Компоновщик (Composite)

Компоновщик – паттерн, структурирующий объекты.

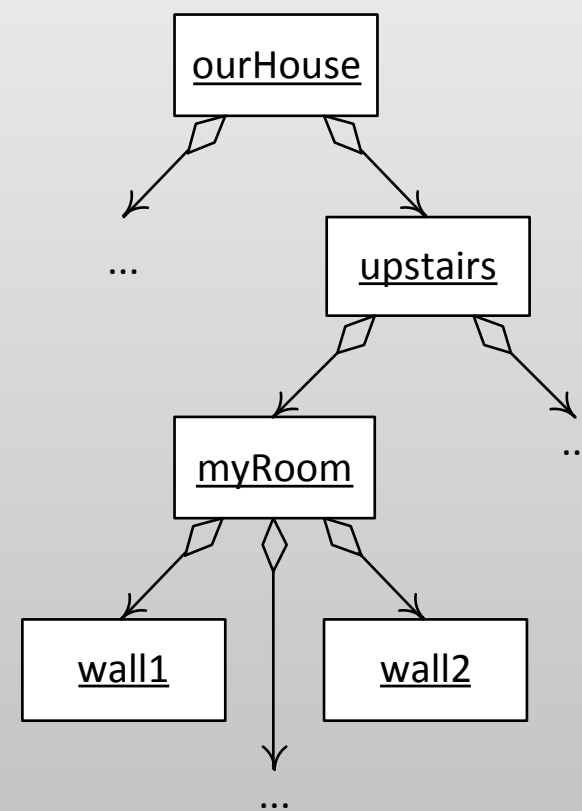
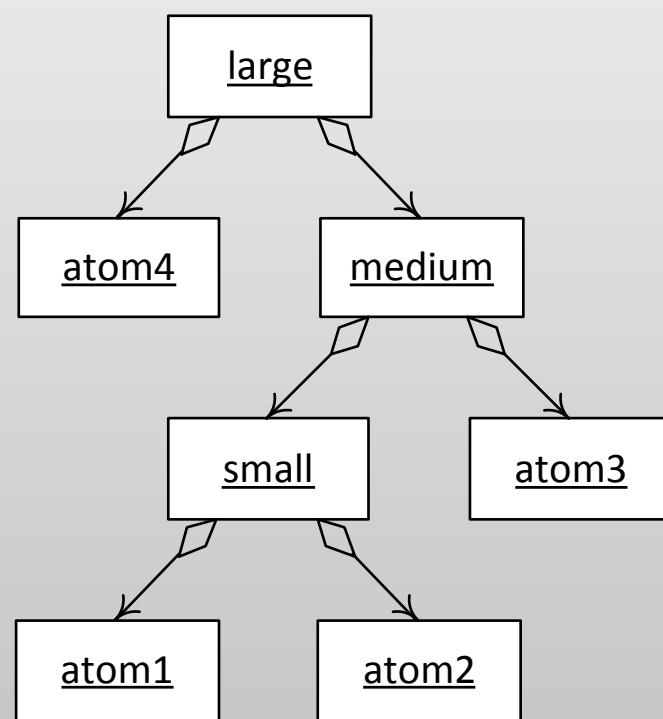
Компонуется объекты в древовидные структуры для представления иерархий «часть-целое».

Позволяет клиентам единообразно трактовать индивидуальные и составные объекты.

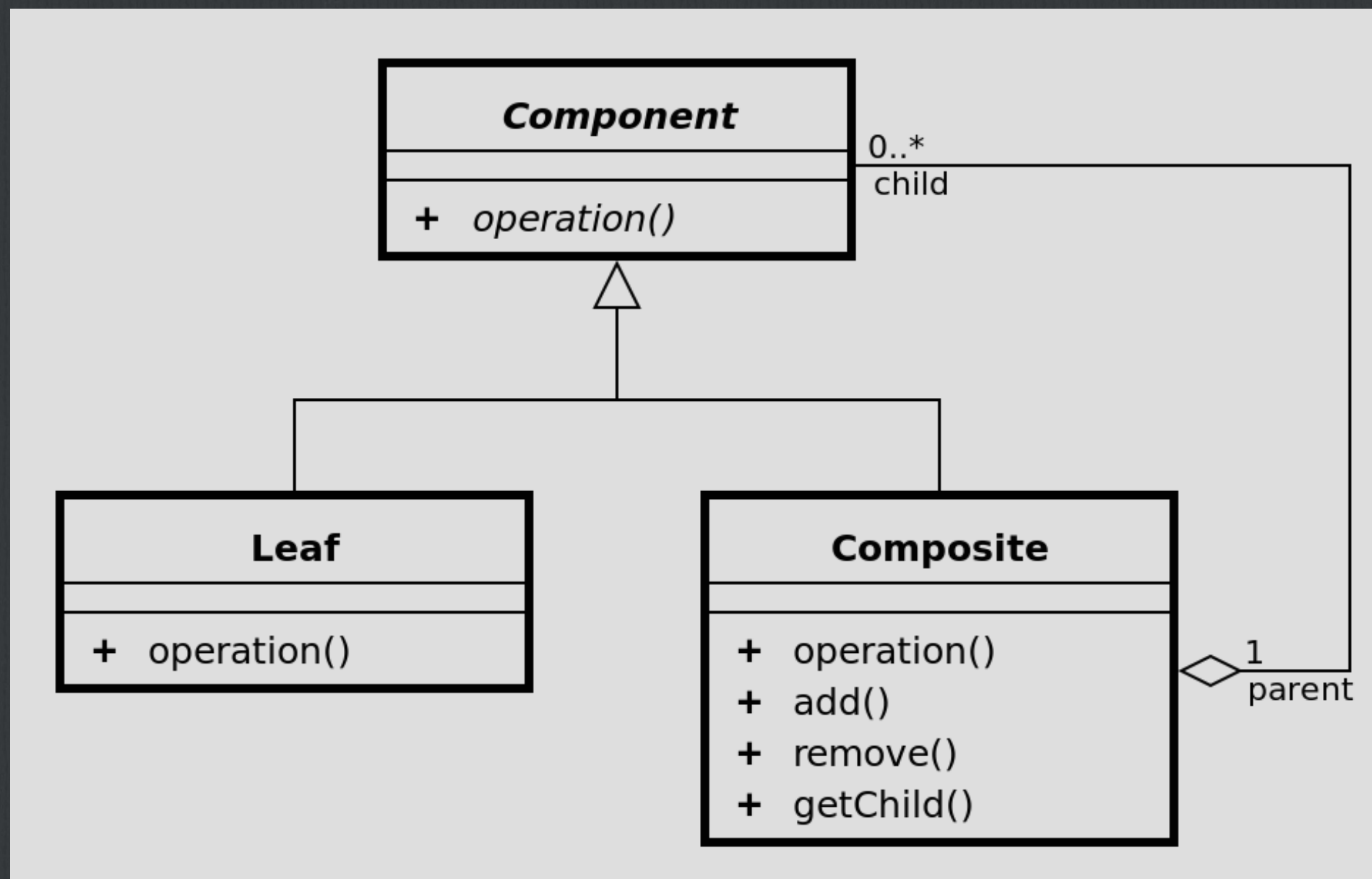
Композитор (Composite)



Композитик (Composite)



Композит (Composite)



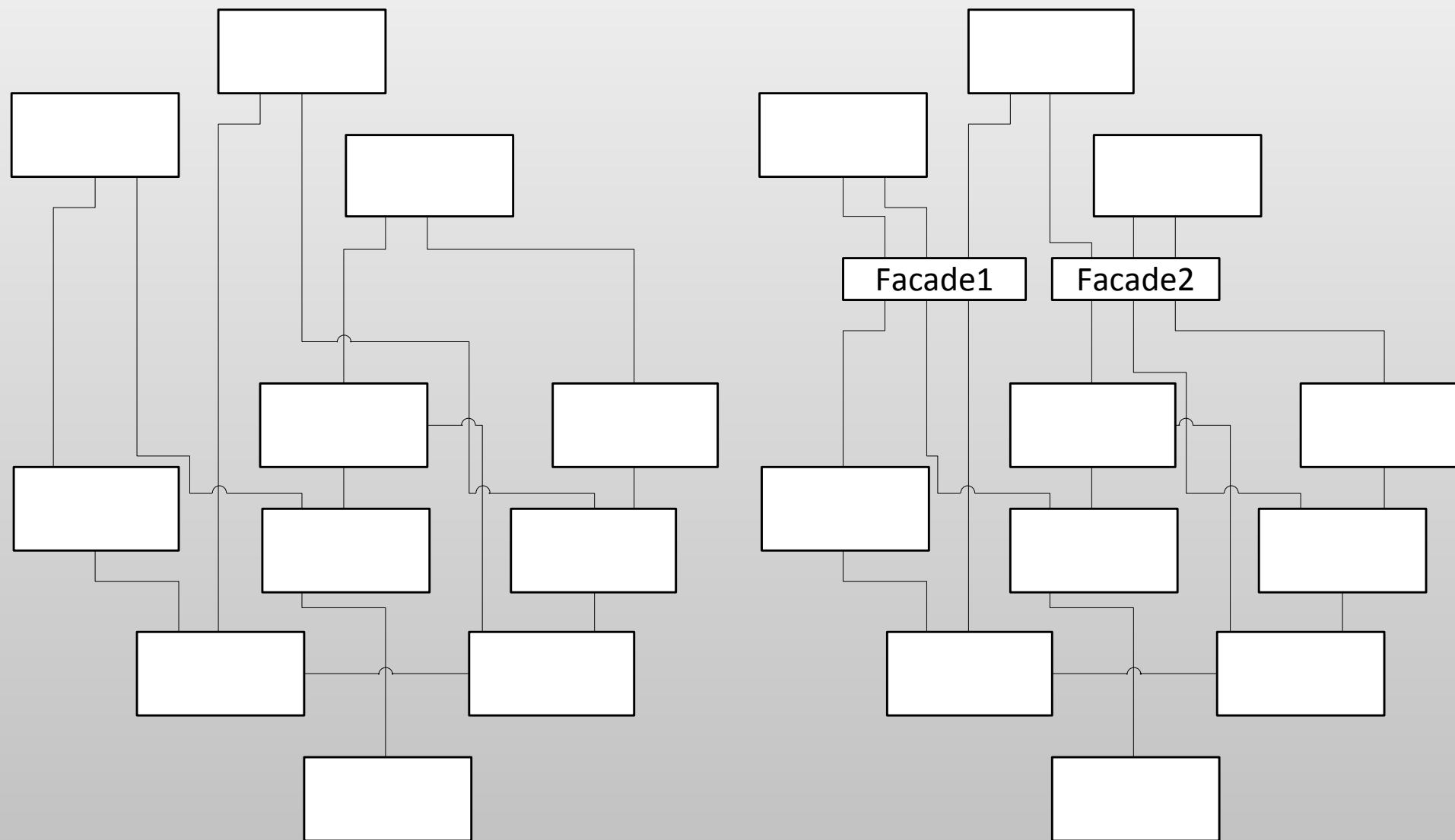
Фасад (Facade)

Фасад – паттерн, структурирующий объекты.

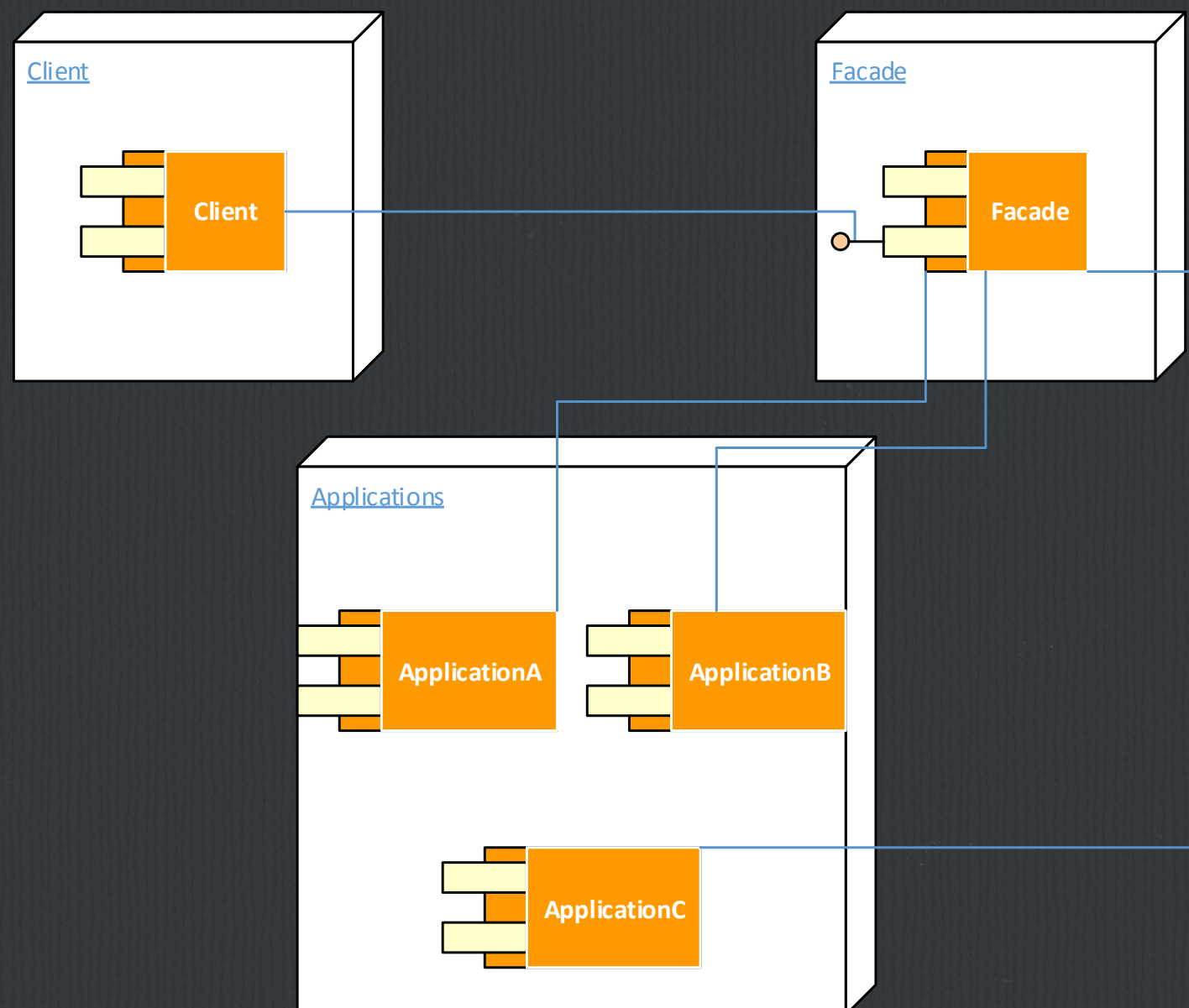
Фасад предоставляет унифицированный интерфейс вместо набора интерфейсов некоторой подсистемы.

Фасад определяет интерфейс более высокого уровня, который упрощает использование подсистемы.

Фасад (Facade)



Фасад (Facade)



Фасад (Facade)

Вся сложность подсистемы и взаимодействие ее компонентов скрыты от клиентов.

"Фасад" переадресует пользовательские запросы подходящим методам подсистемы. Клиент использует только "фасад".

Связь с другими шаблонами – объект «фасада» обычно бывает «одиночкой».

Адаптер (Adapter)

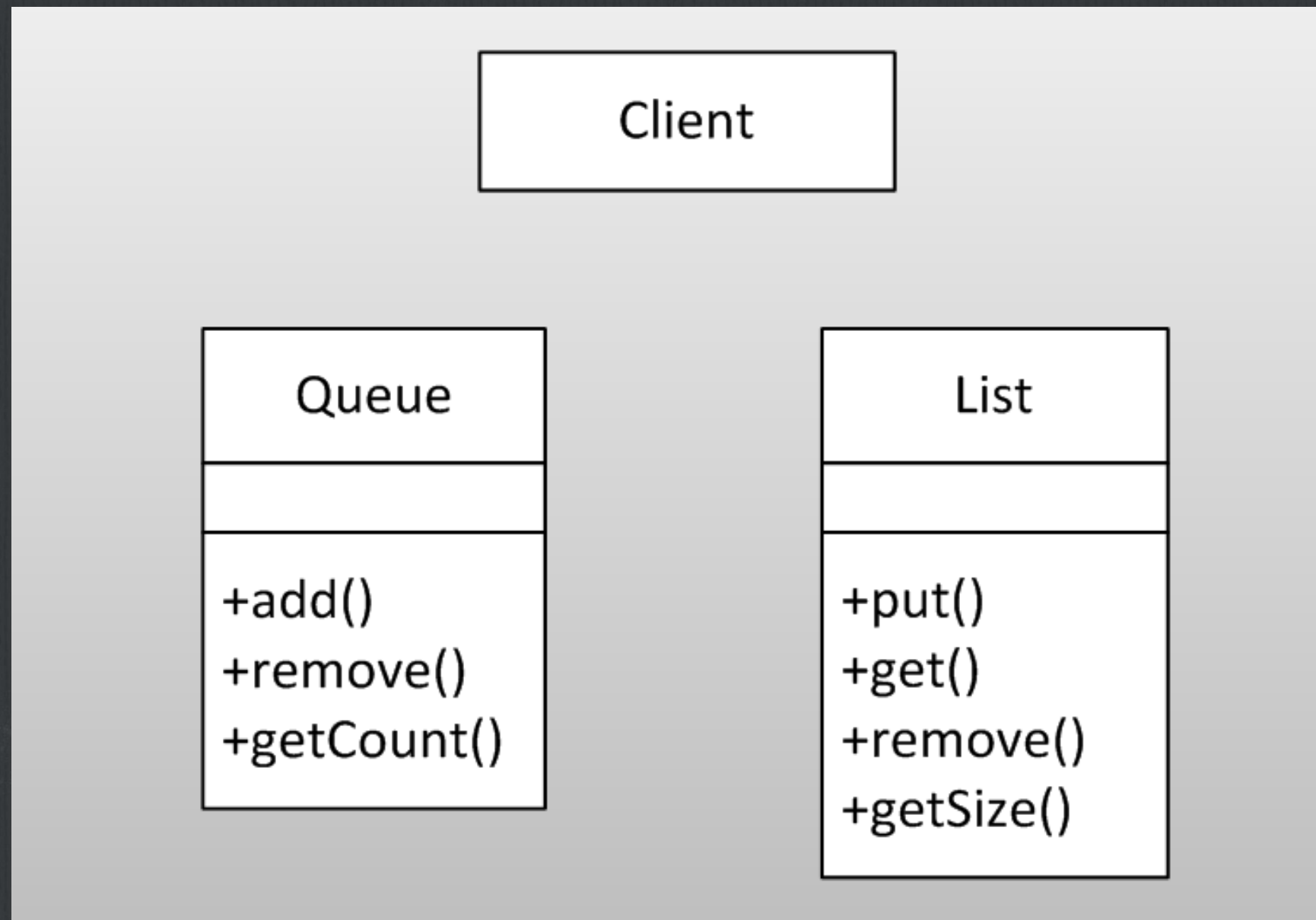
Адаптер – паттерн, структурирующий классы и объекты.

Адаптер преобразует интерфейс одного класса в интерфейс другого, который ожидают клиенты.

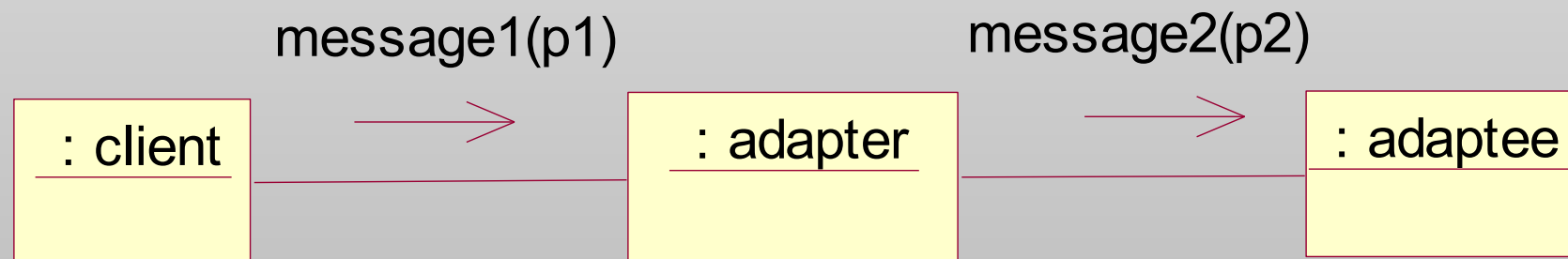
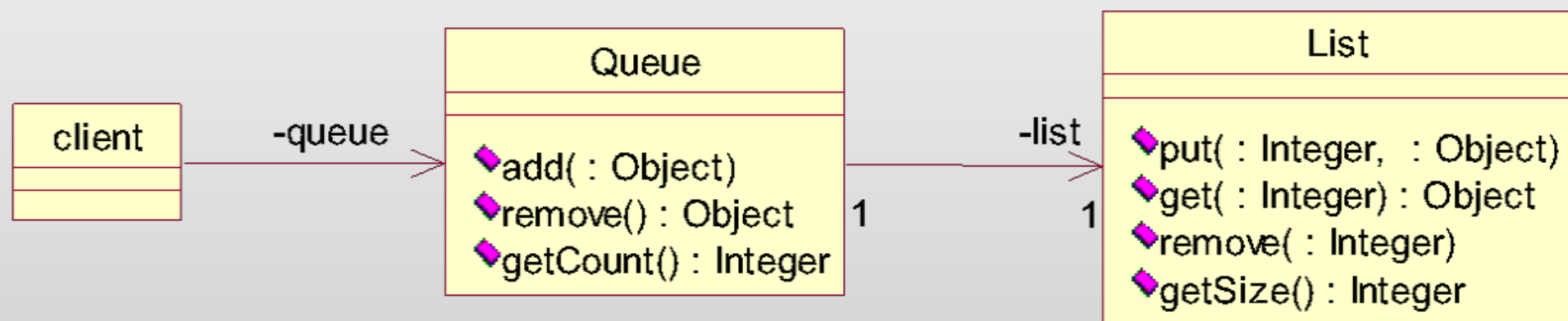
Адаптер обеспечивает совместную работу классов с несовместимыми интерфейсами.

Другое название шаблона – Обёртка (Wrapper).

Адаптер (Adapter)



Адаптер (Adapter)



Адаптер (Adapter)

```
public class Queue {  
    private List list;  
  
    public Queue() {  
        list = new List();  
    }  
  
    public void add (Object o) {  
        list.add(o);  
    }  
  
    public Object remove() {  
        return list.remove(0);  
    }  
  
    public int getCount() {  
        return list.getSize();  
    }  
}
```


Фасад vs. Адаптер

- ✓ фасад определяет новый интерфейс, в то время как адаптер использует уже имеющийся, т.е. адаптер делает работающими вместе два существующих интерфейса, не создавая новых
- ✓ и адаптер , и фасад являются обёртками разных типов. Цель фасада – создать более простой интерфейс, цель адаптера – адаптировать уже существующий. Фасад обычно обёртывает несколько объектов, адаптер только один.

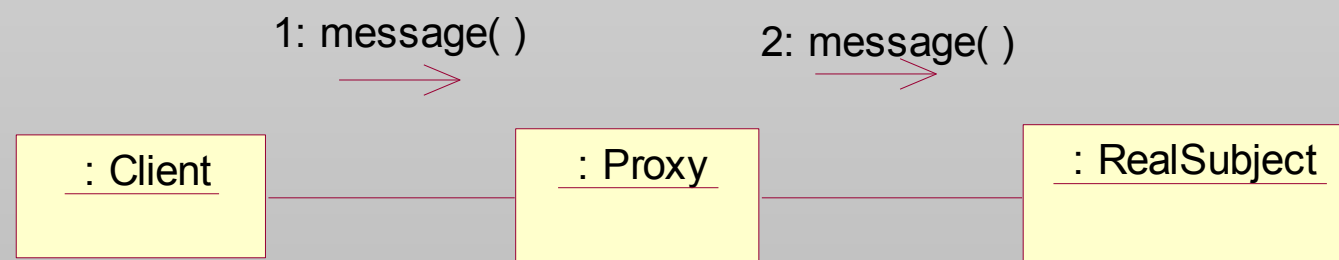
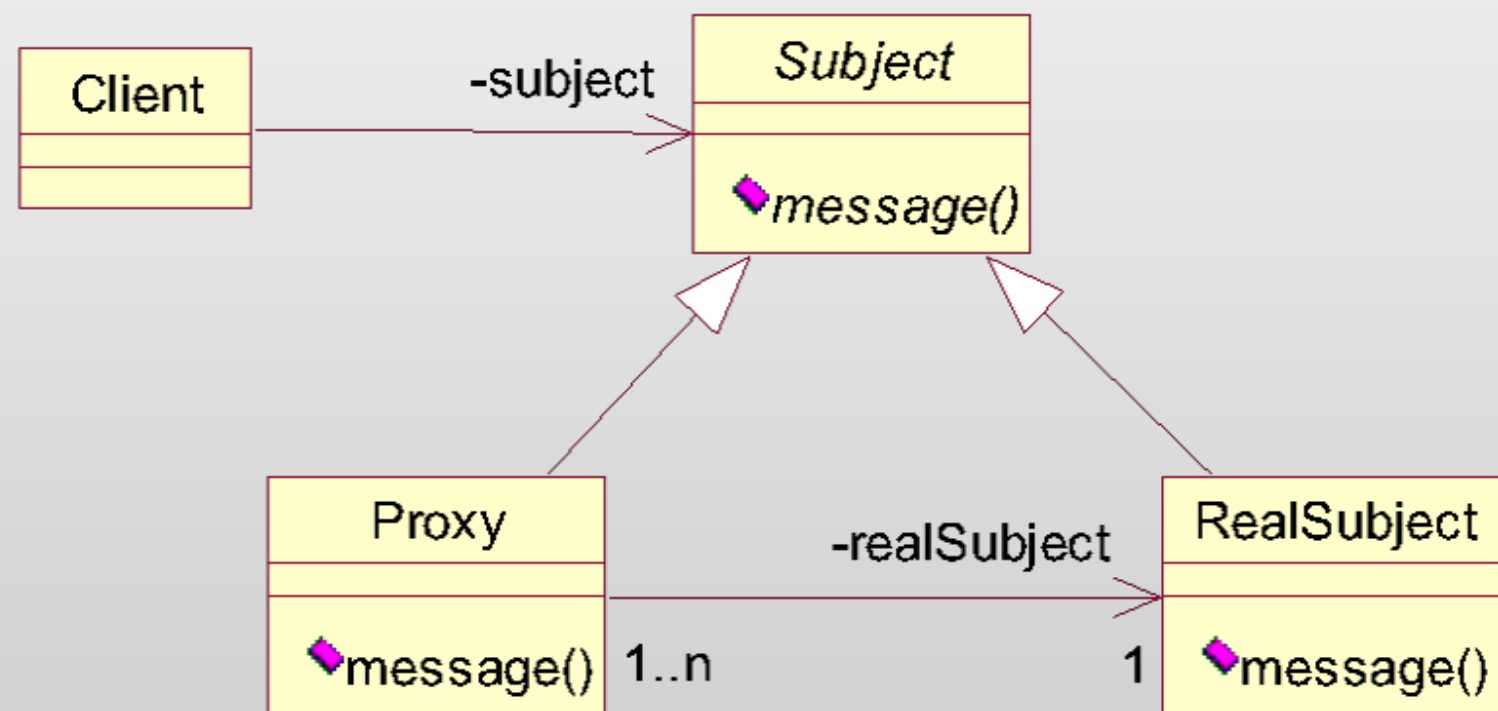
Заместитель (Proxy)

Заместитель – паттерн, структурирующий объекты.

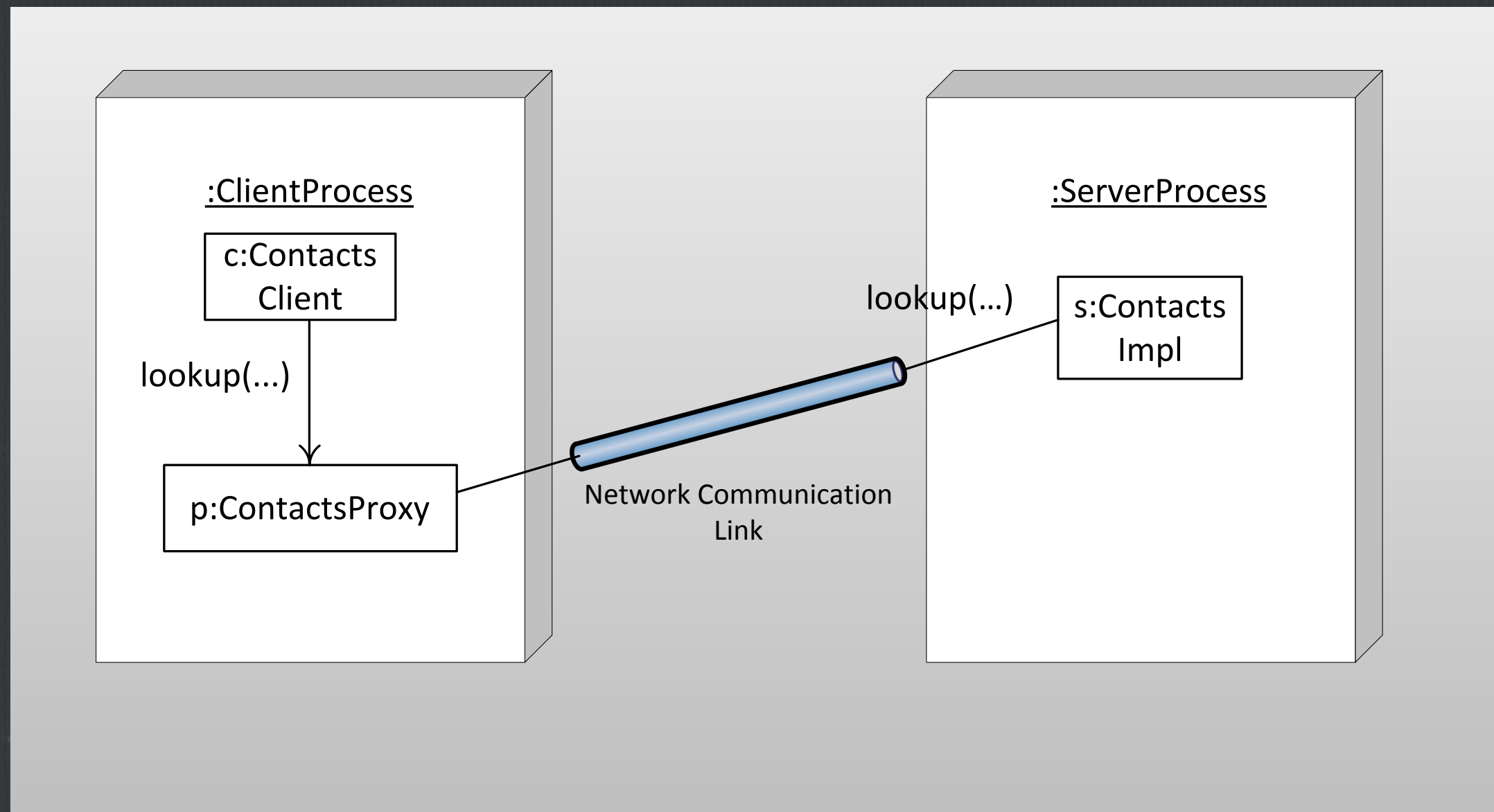
Прокси является суррогатом другого объекта и контролирует доступ к нему.

Объект-прокси временно используется вместо реального объекта (до его создания, если реальный объект слишком тяжеловесен – например, изображение), ведёт себя точно так же как он, и выполняет, если необходимо, его инстанцирование.

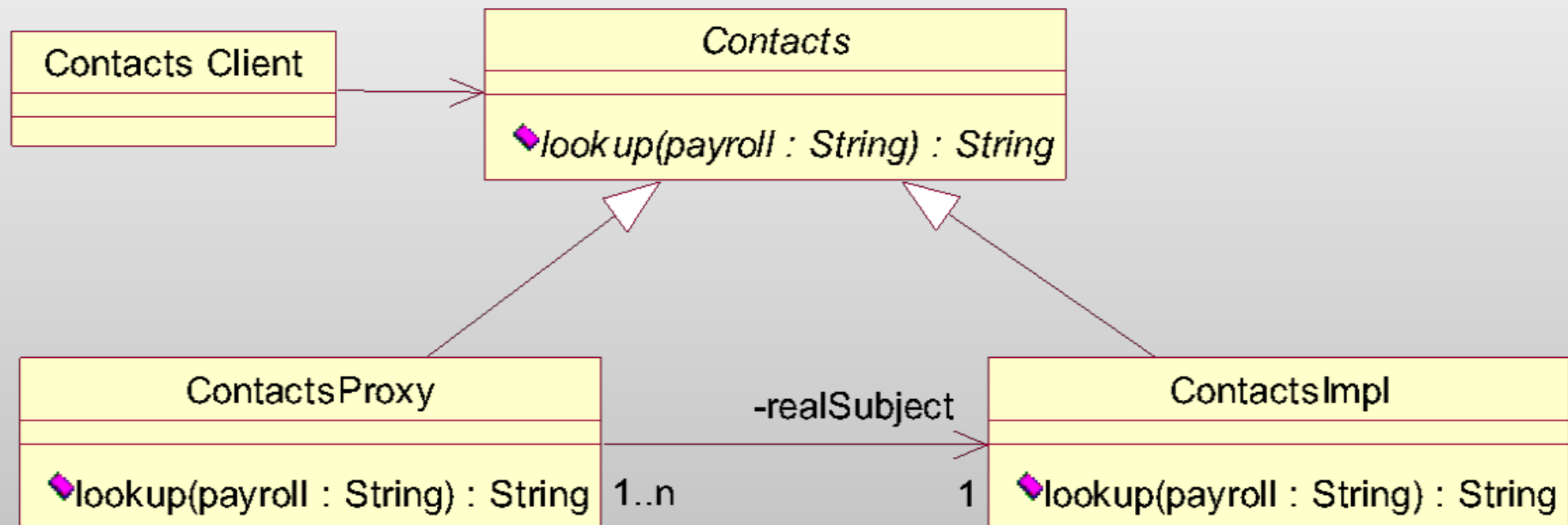
Заместитель (Proxy)



Заместитель (Proxy)



Заместитель (Proxy)

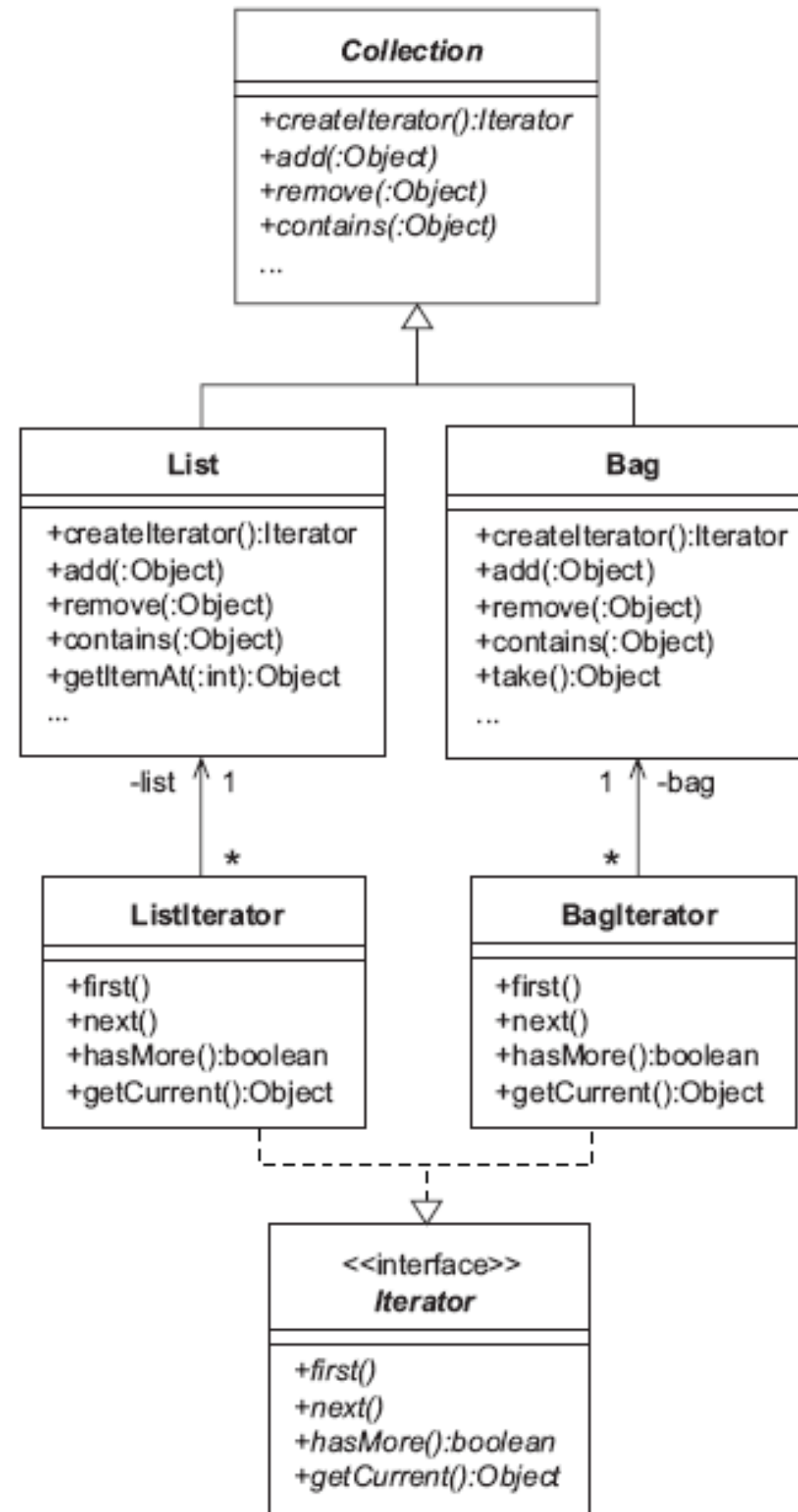


Паттерны поведения

Итератор (Iterator)

Итератор – паттерн поведения объектов.

Назначение: предоставляет способ последовательного доступа ко всем элементам составного объекта, не раскрывая его внутреннего представления.



Iterator (Implementation)

```
public class ListIterator implements
    Iterator{
    private List list;
    private int index = 0;

    protected ListIterator(List l){
        list = l;
    }
    public void first(){
        index = 0;
    }
    public void next(){
        index = index + 1;
    }
    public boolean hasMore(){
        return index < list.getSize;
    }
    public Object getCurrent(){
        return list.getItemAt(index);
    }
}
```


Observer (Наблюдатель)

Car Search - 0 x

Color

Red ▼

Doors

2 ▼

CD Player

Yes ▼



Price

\$36,5

Car Search - 0 x

Color

Yellow ▼

Doors

2 ▼

CD Player

Yes ▼



Price

\$34,5

Observer (Наблюдатель)

1. Название

Наблюдатель (Observer)

2. Классификация

Шаблон поведения объектов

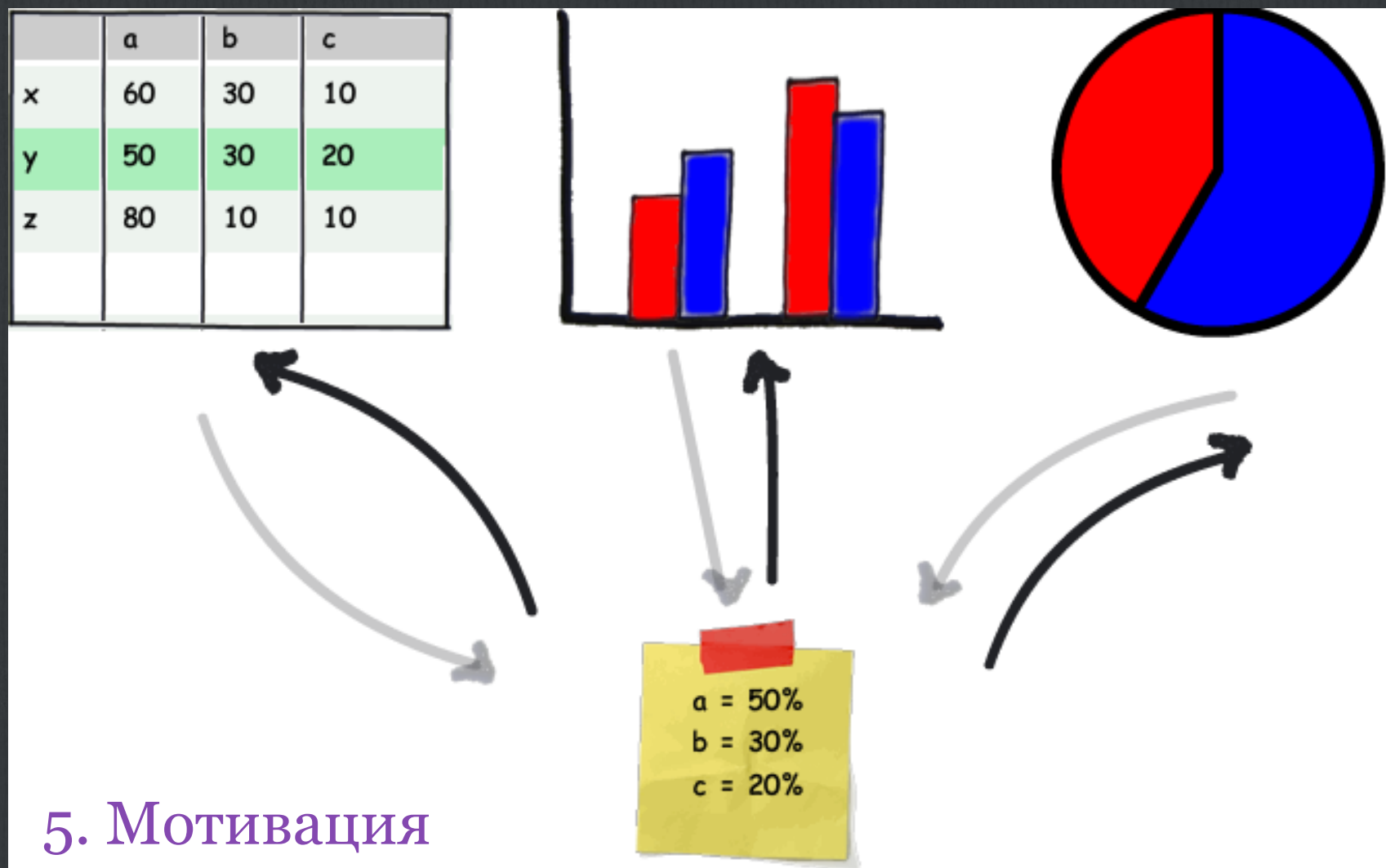
3. Назначение

Определяет зависимость между объектами типа «один ко многим», таким образом, что при изменении состояния одного объекта все зависящие от него объекта оповещаются об этом факте и изменяют своё состояние.

4. Известен также

Dependents (подчинённые), publisher-subscriber (издатель-подписчик)

Observer (Наблюдатель)



5. Мотивация

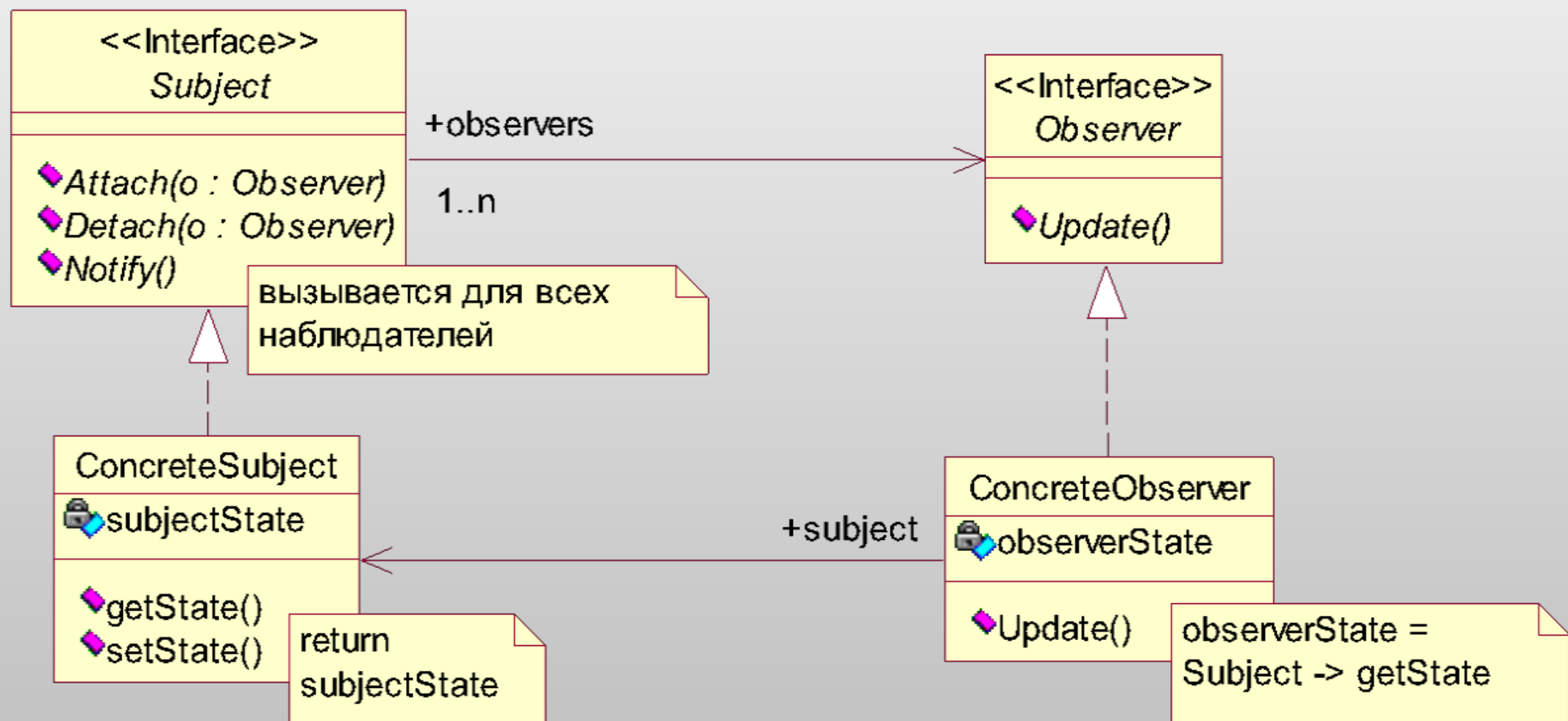
Observer (Наблюдатель)

6. Применение

когда при модификации одного объекта нужно изменять другие, и мы точно не знаем сколько объектов нужно изменять

когда один объект должен оповещать другие, не делая предположений об уведомляемых объектах

Observer (Наблюдатель)



7. Структура

Observer (Наблюдатель)

8. Участники

Subject – субъект:

- располагает информацией о своих наблюдателях, которых может быть любое число
- предоставляет интерфейс для присоединения/отсоединения наблюдателей

Observer – наблюдатель:

- определяет интерфейс обновления объектов, которые должны быть уведомлены об изменении состояния субъекта

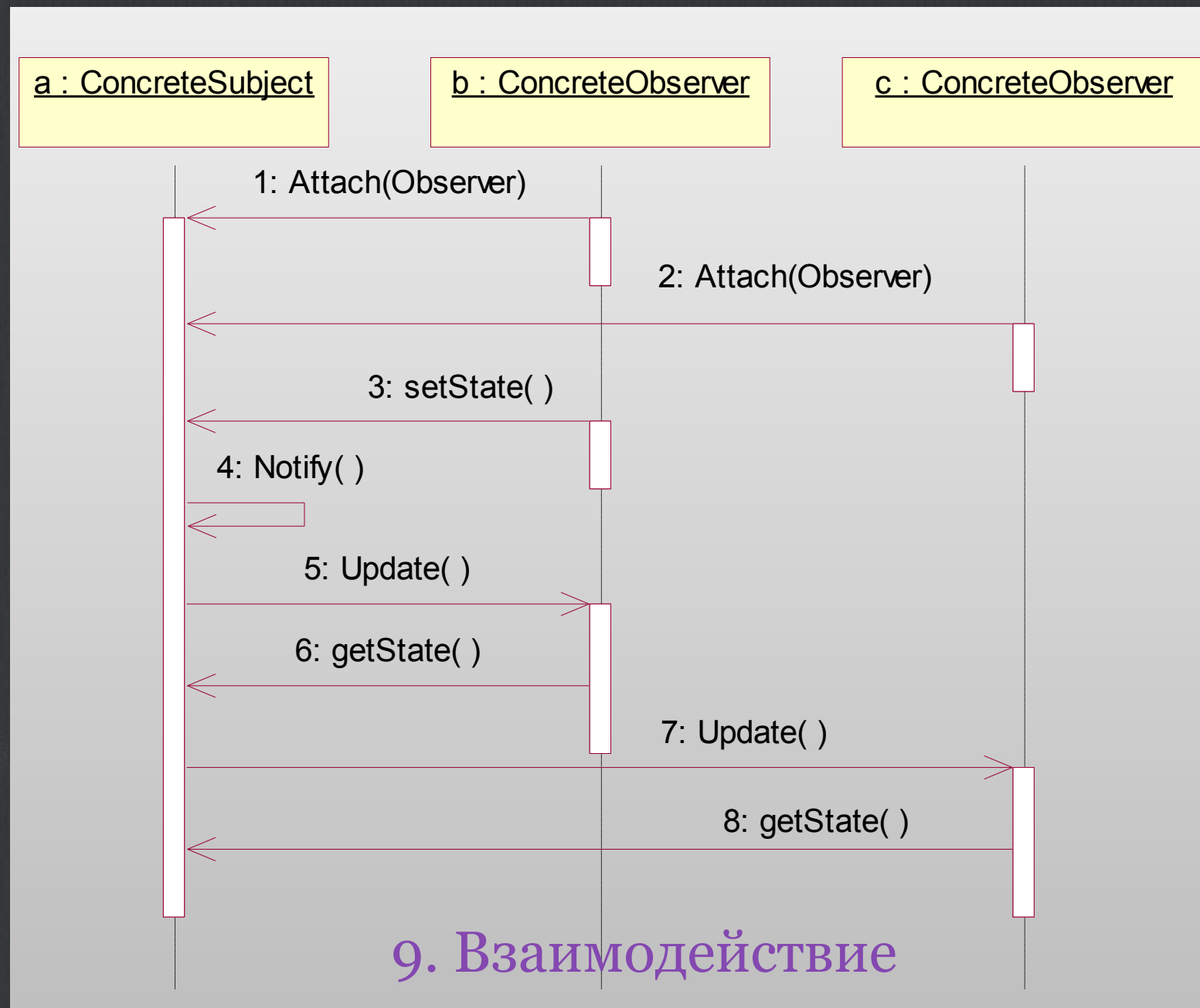
ConcreteSubject - конкретный субъект:

- сохраняет состояние, представляющее интерес для конкретного наблюдателя
- посылает информацию своим наблюдателям, когда происходит изменение

ConcreteObserver – конкретный наблюдатель

- хранит ссылку на объект конкретного субъекта
- сохраняет данные, которые должны быть согласованы с данными субъекта
- реализует интерфейс обновления

Observer (Наблюдатель)



Observer (Наблюдатель)

10. Последствия

- + абстрактная связанность субъекта и наблюдателя
 - + поддержка широковещательных коммуникаций
- неожиданные обновления

Observer (Наблюдатель)

11. Реализация

- отображение субъектов на наблюдателей
- наблюдение более чем за одним субъектом
- инициатор обновления (субъект или объект)
- висячие ссылки на удалённые субъекты
- гарантии непротиворечивости состояния субъекта перед отправкой уведомления
- явное специфицирование представляющих интерес модификаций

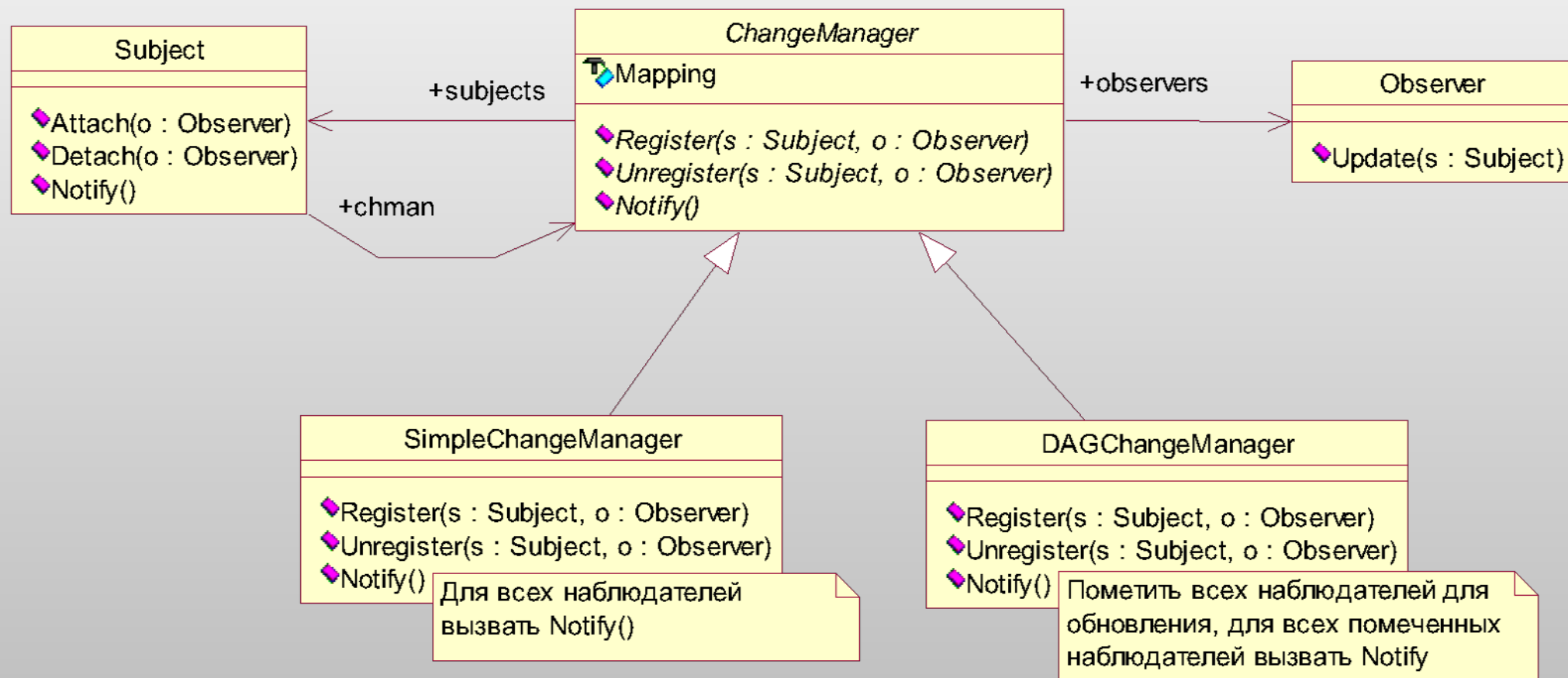
```
void Subject::Attach(Observer*, AspectInterest);
```

```
void Observer::Update(Subject*, AspectInterest);
```


Observer (Наблюдатель)

11. Реализация

- инкапсуляция сложной семантики обновления



Observer (Наблюдатель)

13. Известные применения

MVC

14. Родственные шаблоны

- Посредник (Mediator) – класс ChangeManager действует как посредник между субъектом и наблюдателями, инкапсулируя сложную семантику обновления
- Одиночка (Singleton) – класс ChangeManager может использовать шаблон Singleton, чтобы гарантировать уникальность и глобальную доступность менеджера изменений.

State (Состояние)

Состояние – паттерн поведения объектов.

Назначение: Позволяет объекту варьировать своё поведение в зависимости от внутреннего состояния. Извне может создаваться впечатление, что изменился класс объекта.

Применяем State когда:

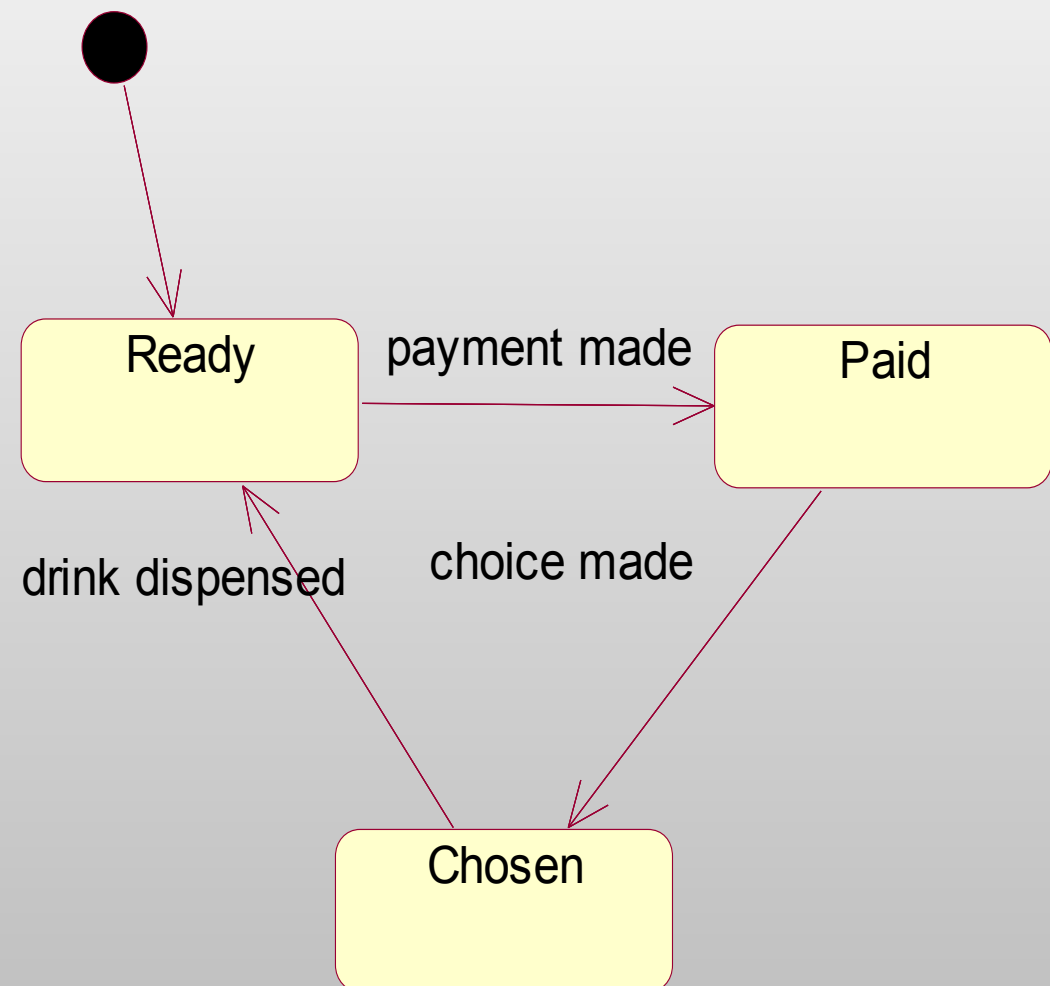
- поведение объекта зависит от его состояния и меняется во время выполнения**
- когда в коде операций встречается много условных операторов, в которых выбор ветви зависит от состояния; часто структура одного и того же оператора повторяется – выносим каждую ветвь в отдельный класс, а состояние объекта трактуем как самостоятельный объект.**

State (Состояние)

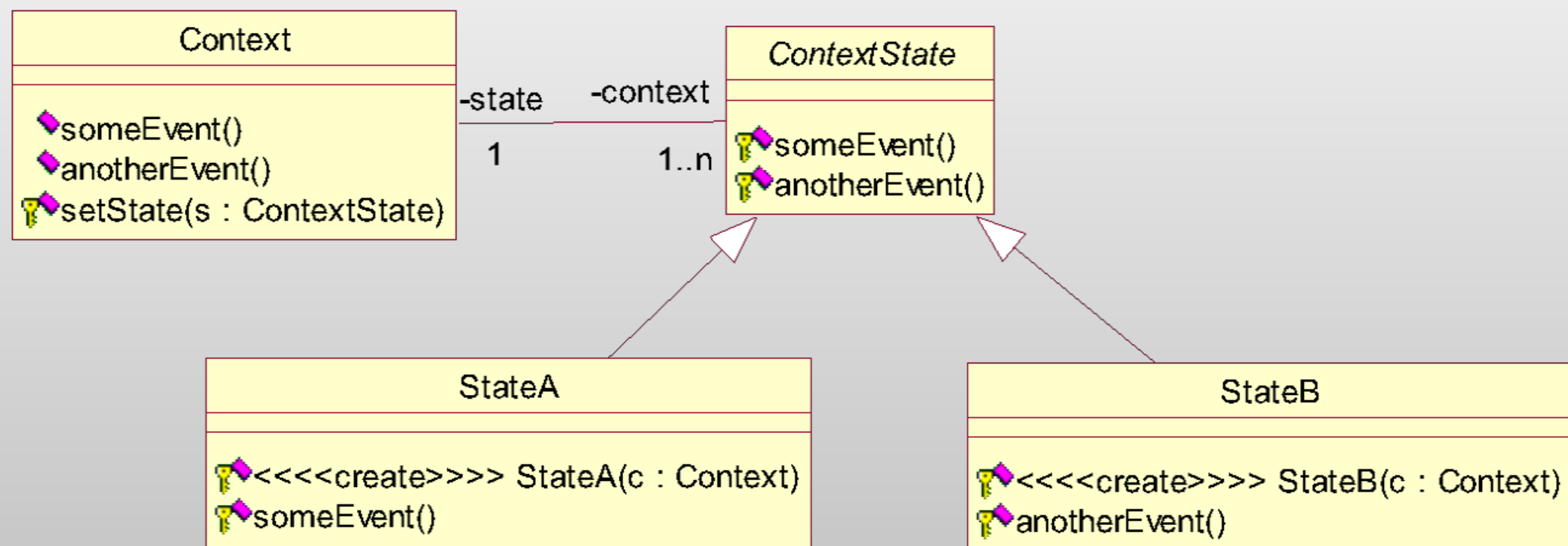
aCoffeeMachine

drinkPrices
availableDrinks
drinkRecipes

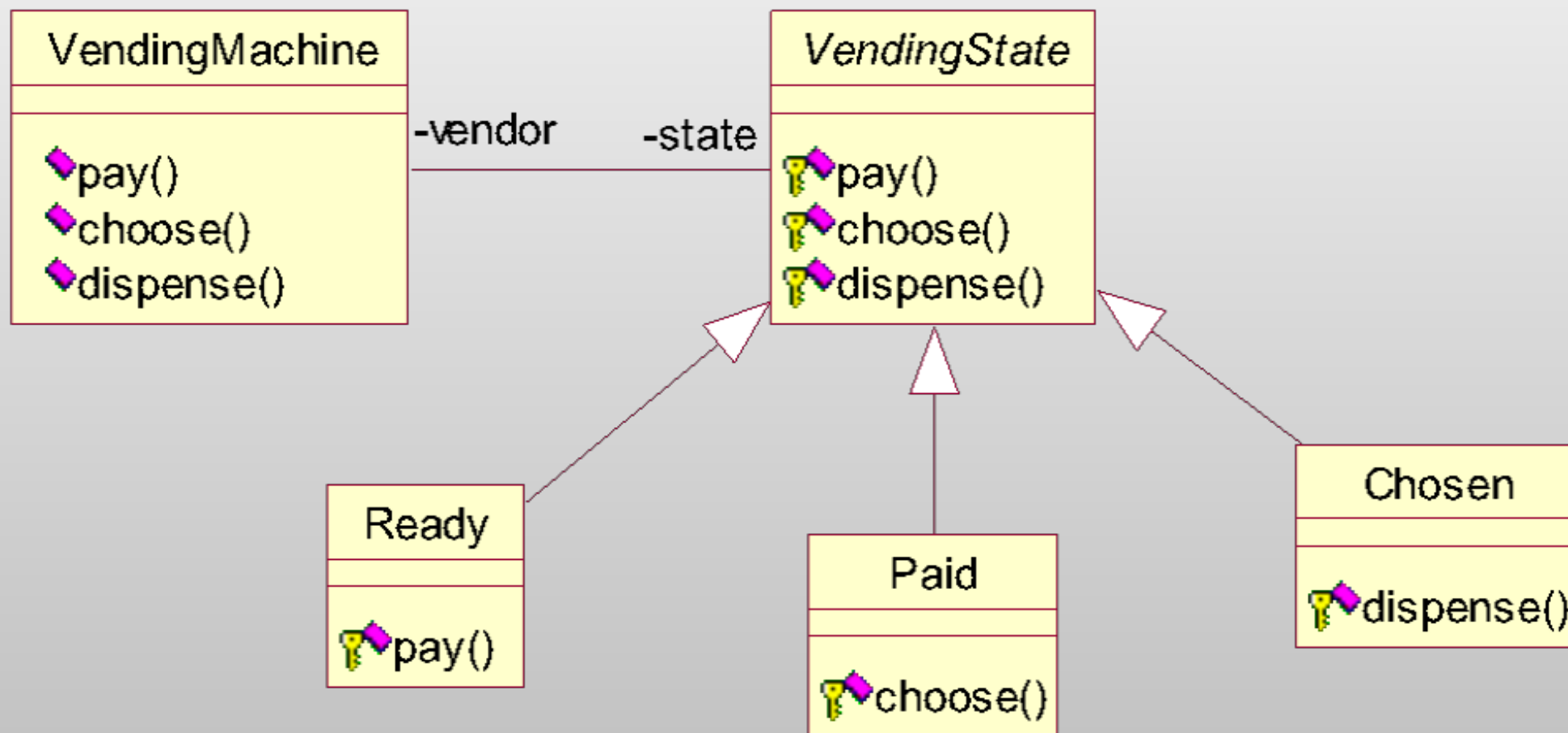
displayDrinks()
selectDrink()
dispenseDrink()
acceptMoney()

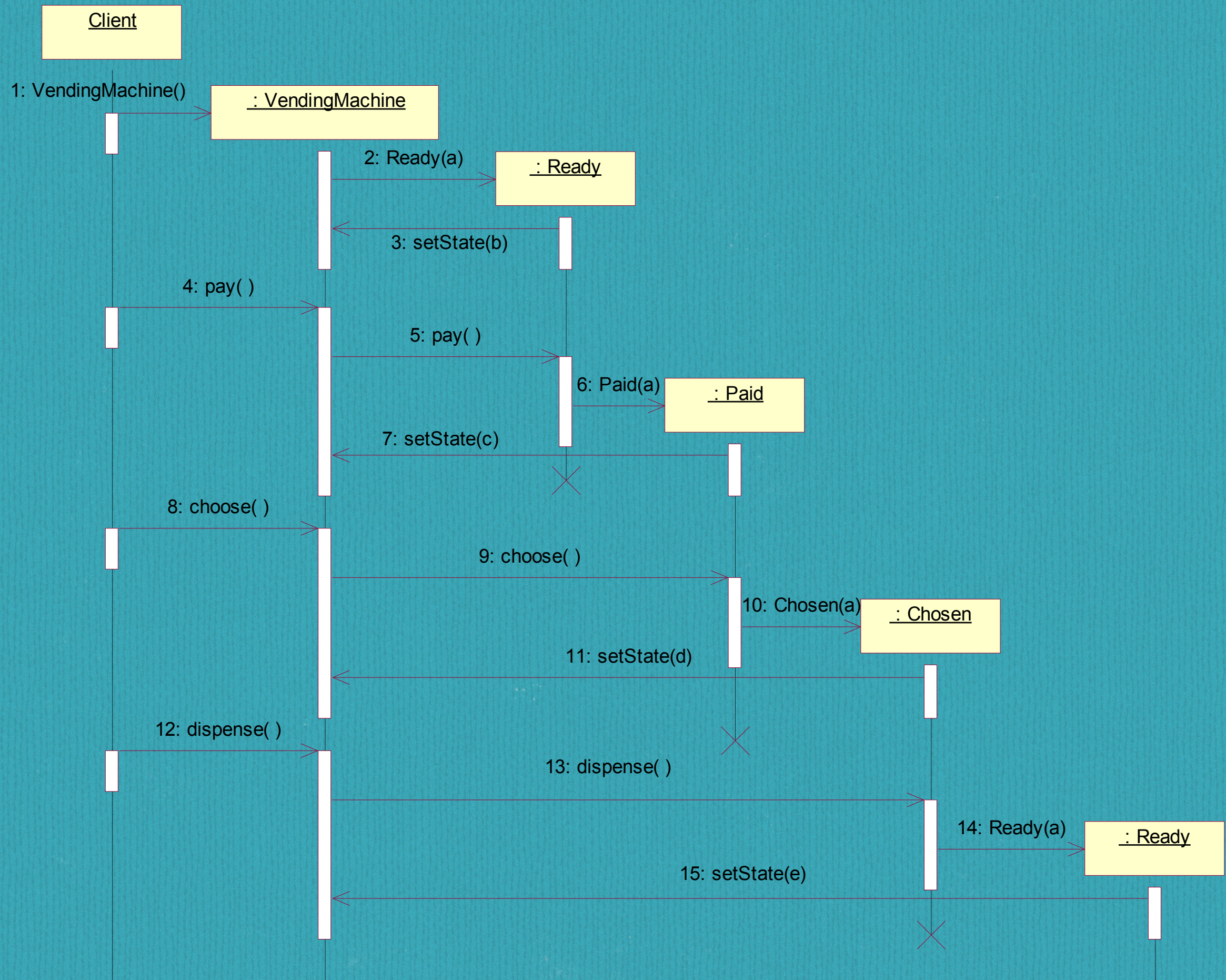


State (Состояние)



State (Состояние)





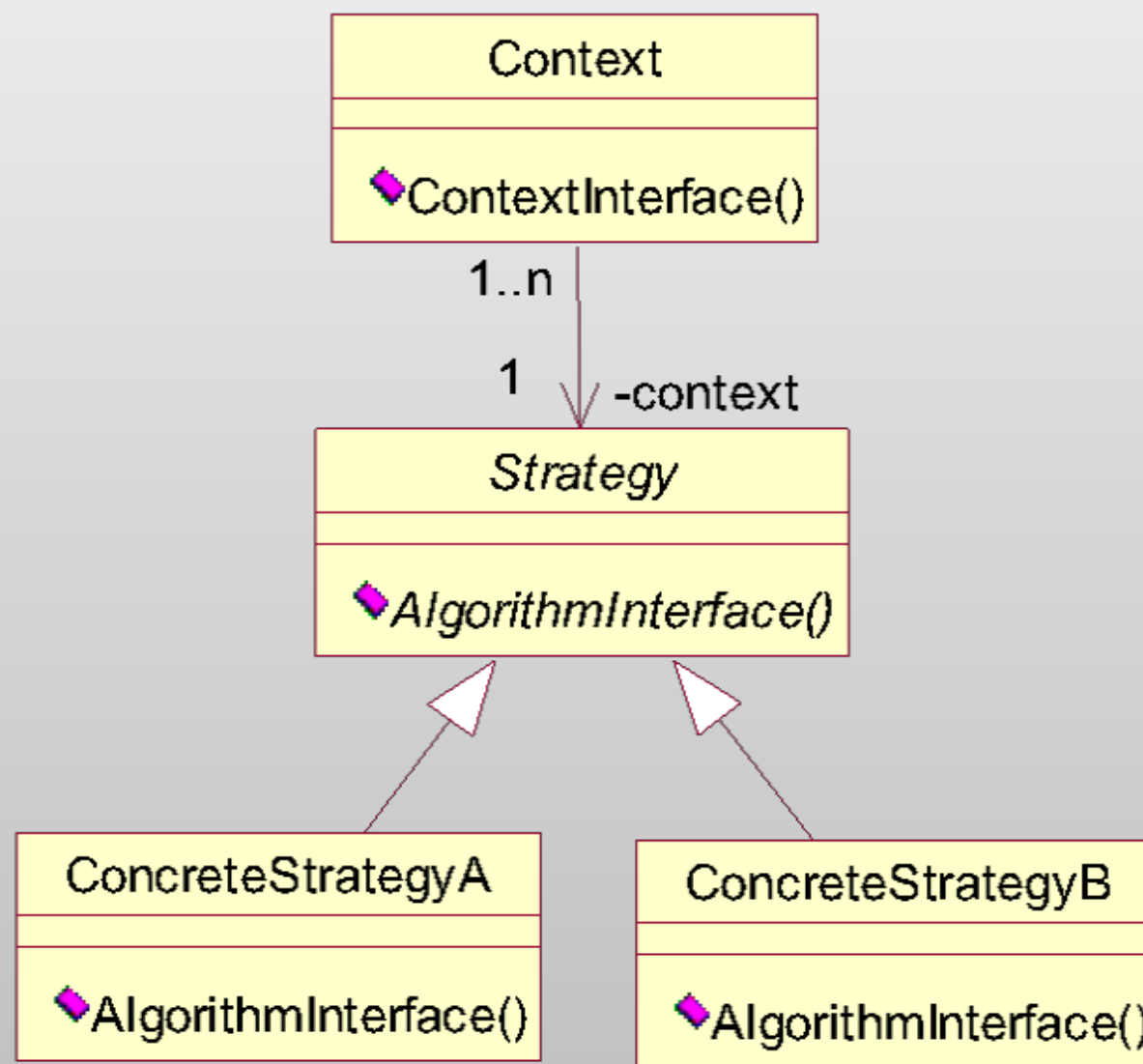
Стратегия (Strategy)

Стратегия – паттерн поведения объектов.

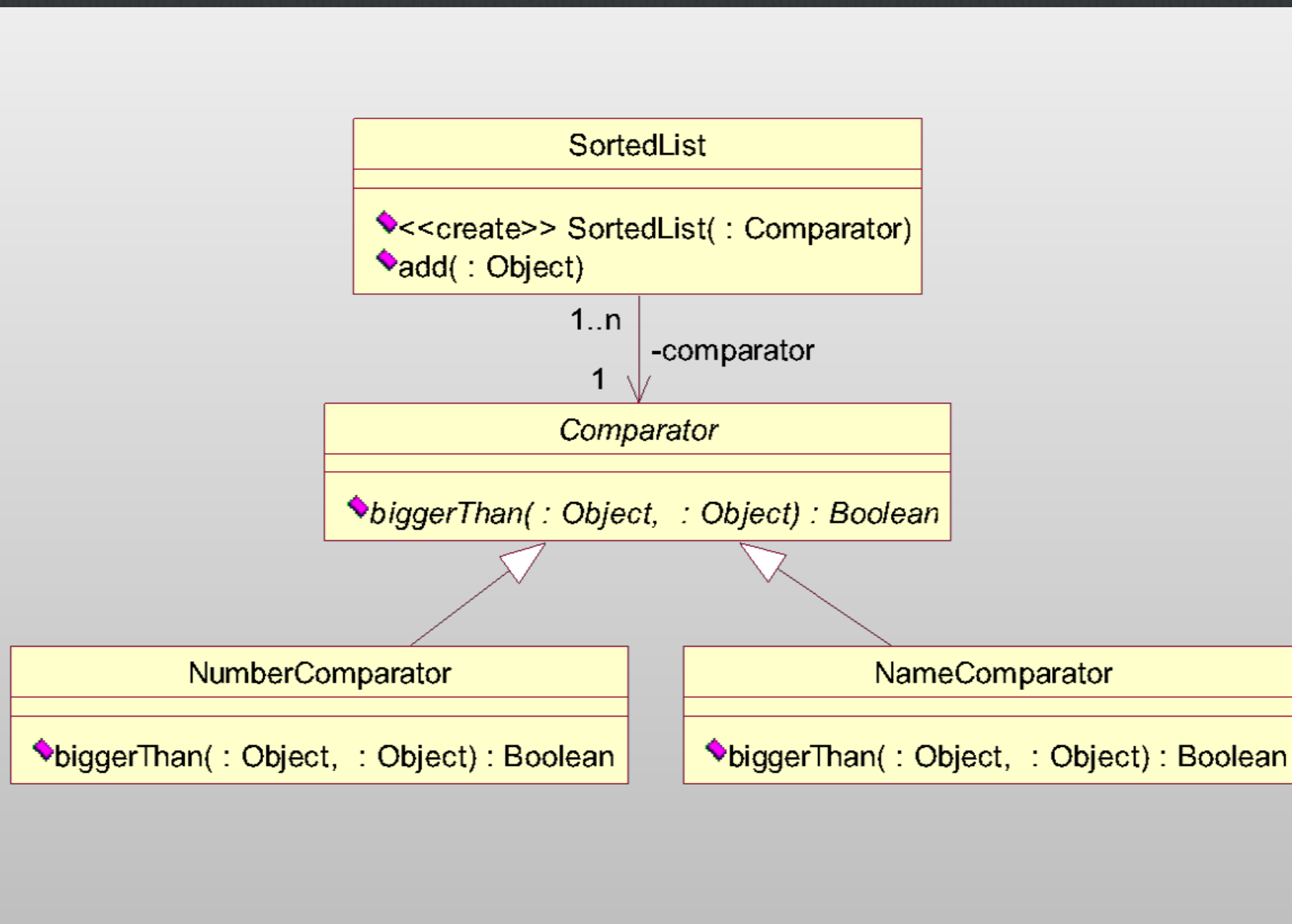
Стратегия определяет семейство алгоритмов, инкапсулирует каждый из них и делает их взаимозаменяемыми.

Стратегия позволяет изменять алгоритмы независимо от клиентов, которые ими пользуются.

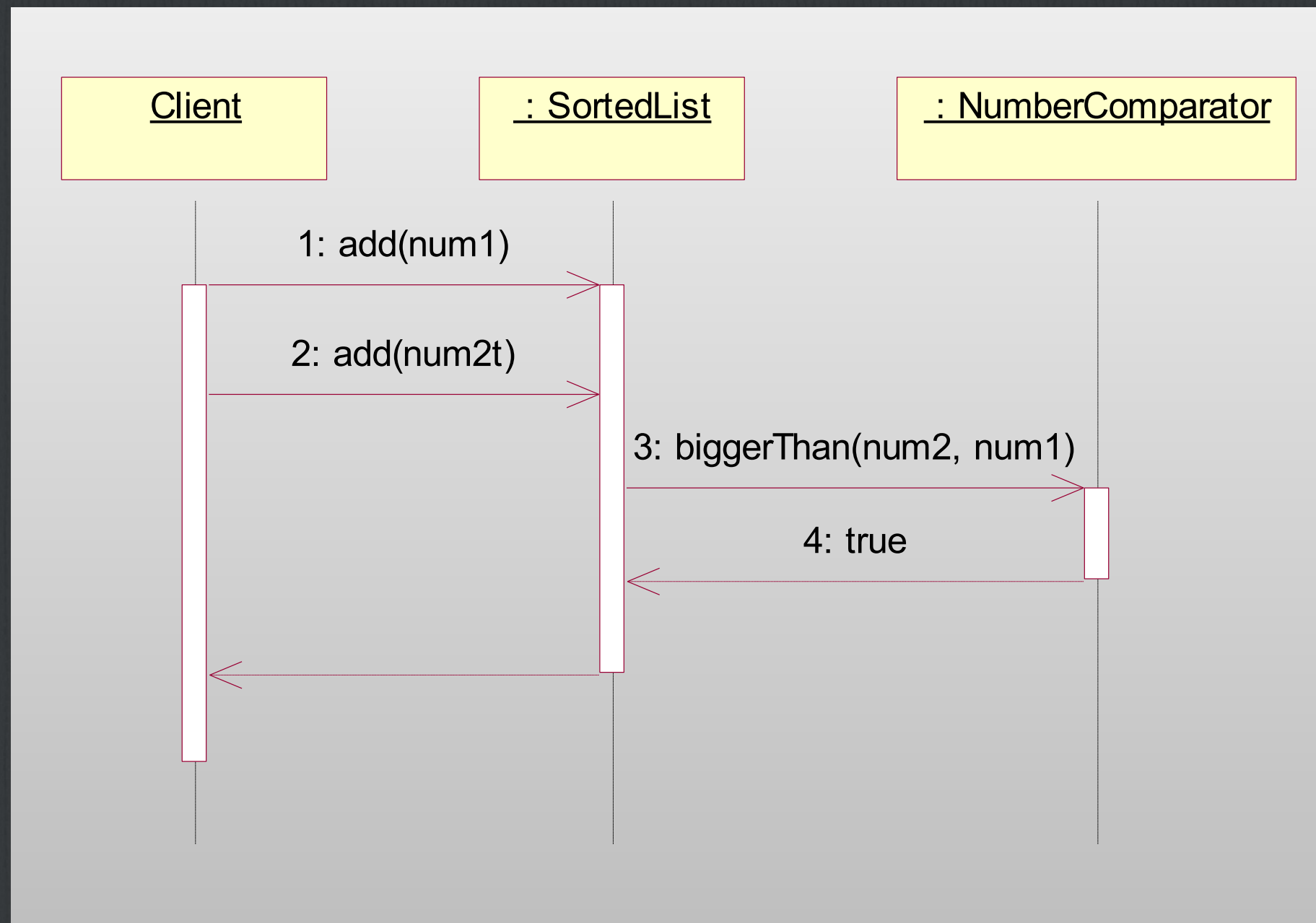
Стратегия (Strategy)



Стратегия (Strategy)



Стратегия (Strategy)



Порождающие паттерны

Одиночка (Singleton)

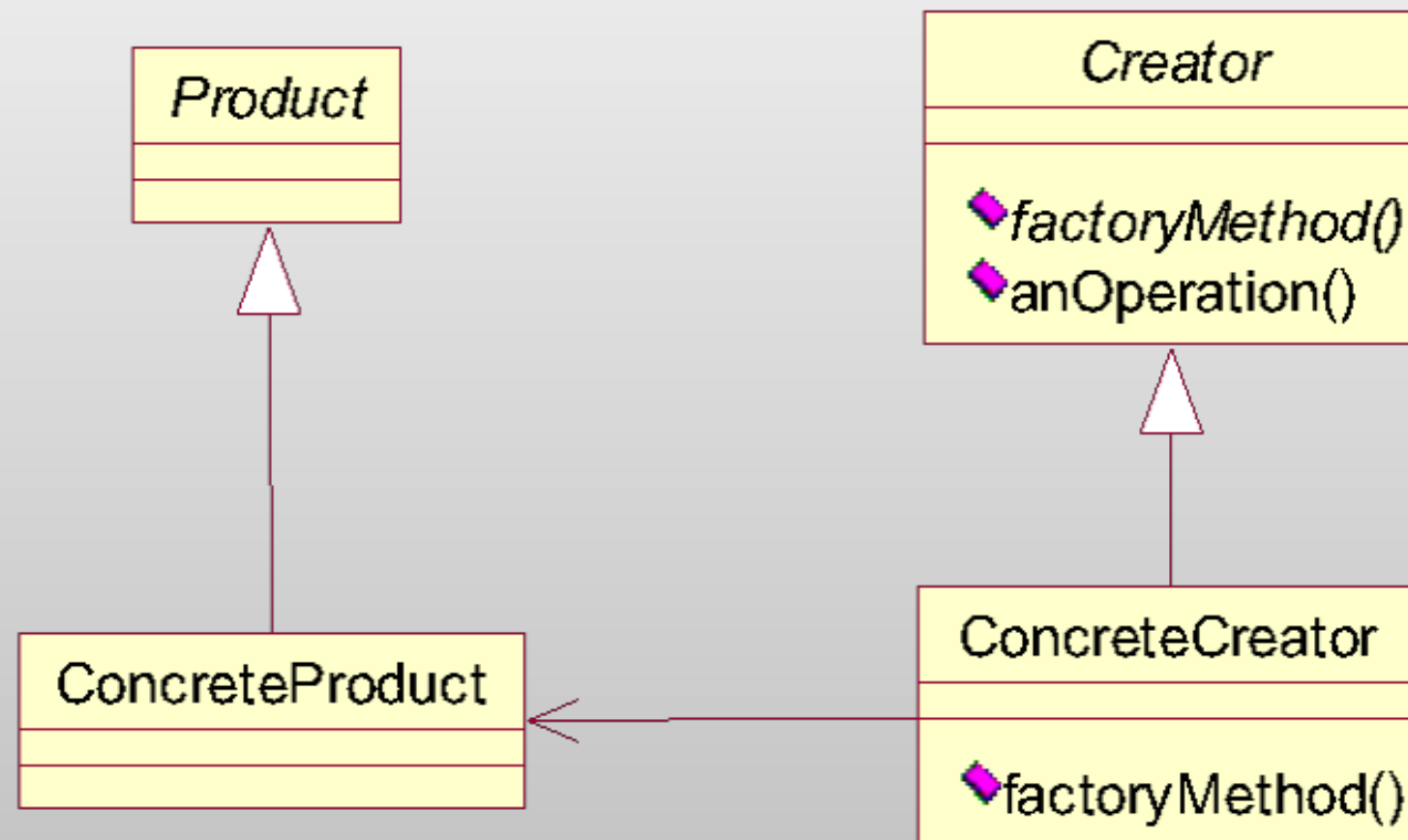


Фабричный метод и абстрактная фабрика

Фабричный метод (Factory Method) – паттерн, порождающий классы.

Фабричный метод определяет интерфейс для создания объекта, но оставляет подклассам принимать решение о том, какой класс инстанцировать.

Фабричный метод (Factory Method)



Фабричный метод (Factory Method)

Принцип инверсии зависимостей:

«Код должен зависеть от абстракций, а не от конкретных классов»

ссылки на конкретные классы не должны храниться в переменных (при использовании `new` сохраняется ссылка на конкретный класс, при использовании фабрики этого не происходит)

в архитектуре не должно быть классов, производных от конкретных (наследование от конкретного класса приводит к зависимости от него, нужно наследовать от абстрактных классов или интерфейсов)

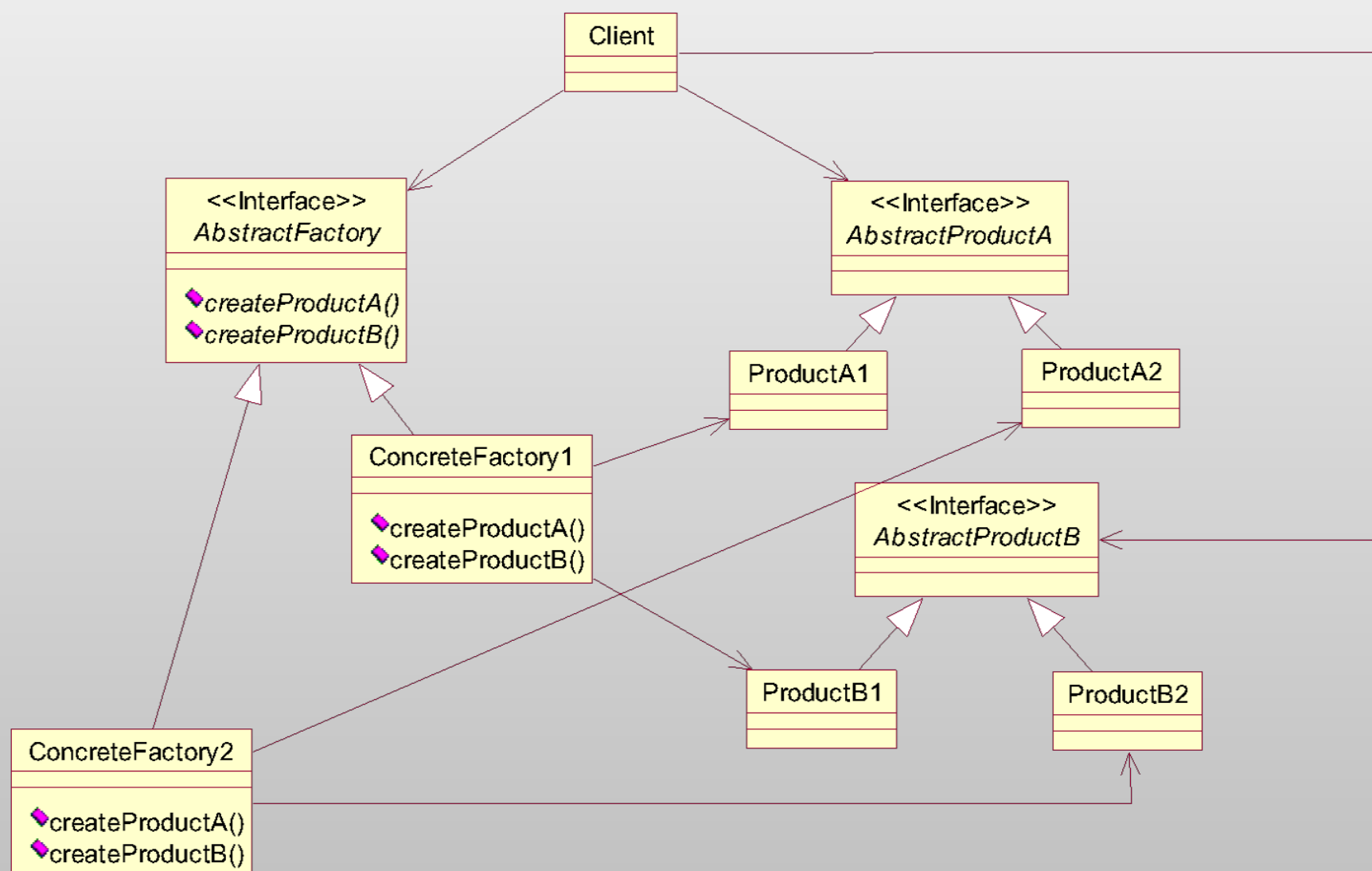
методы класса не должны переопределять методы, реализованные в каких-либо из базовых классов (иначе, базовый класс был плохой абстракцией)

Абстрактная фабрика (Abstract Factory)

Паттерн Абстрактная фабрика предоставляет интерфейс для создания семейств взаимосвязанных или взаимозависимых объектов без указания их конкретных классов.

Методы Абстрактной фабрики часто реализуются как Фабричные методы.

Абстрактная фабрика (Abstract Factory)



Применение паттернов

☐ контекст

☐ задача

☐ решение